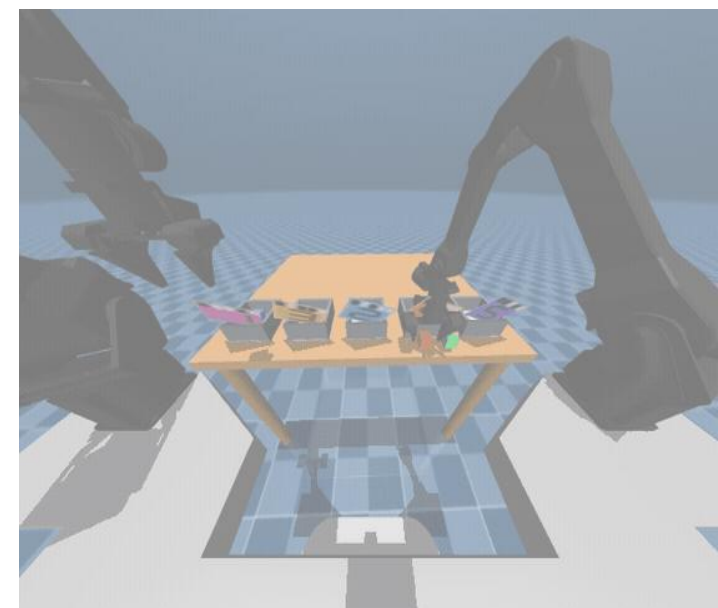


Advantages of CaP in Dynamic Embodied Tasks

CaP / HoloAgent-0 Local / PhyAgentOS Local 对比实验

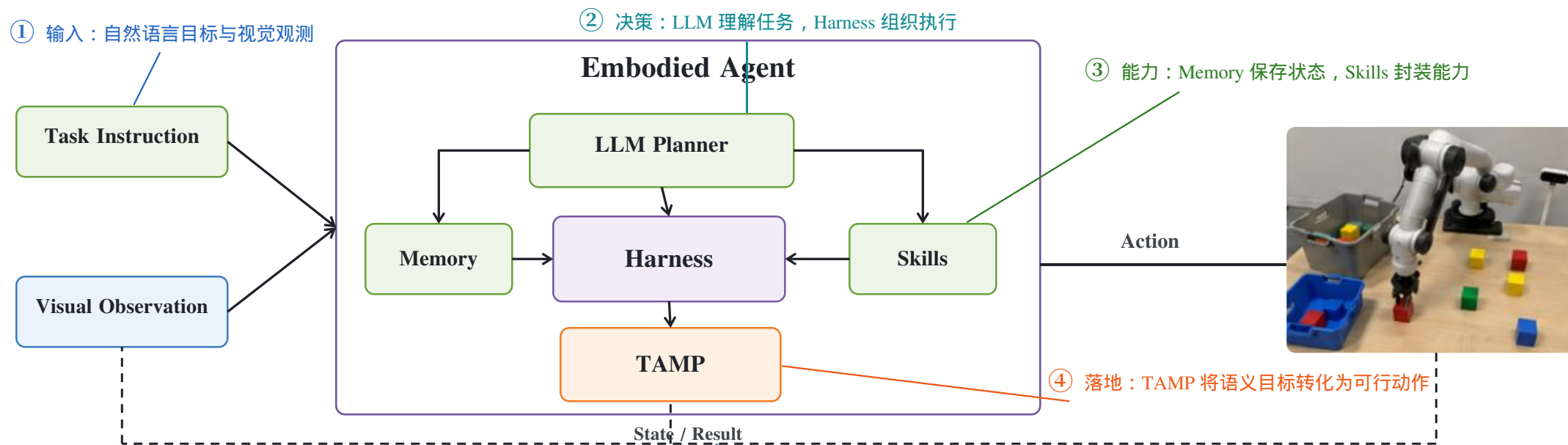
东北大学 郑德轩



OUTLINE

- **1. Embodied Agent Architecture**
- **2. CaP: Online Code Generation & Execution**
- **3. Comparative Experiments on Dynamic Tasks**
- **4. Experimental Results, Contributions & Limitations**

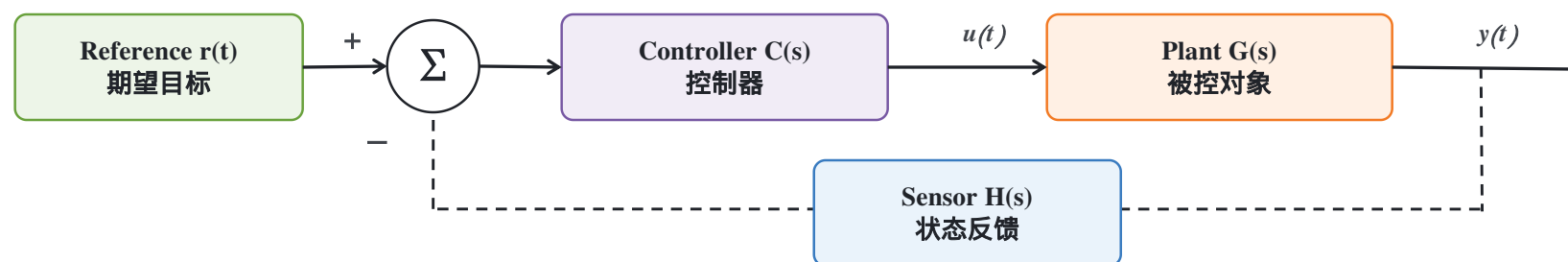
Embodied Agent Architecture



⑤ 闭环：执行结果返回 Agent，驱动下一步决策

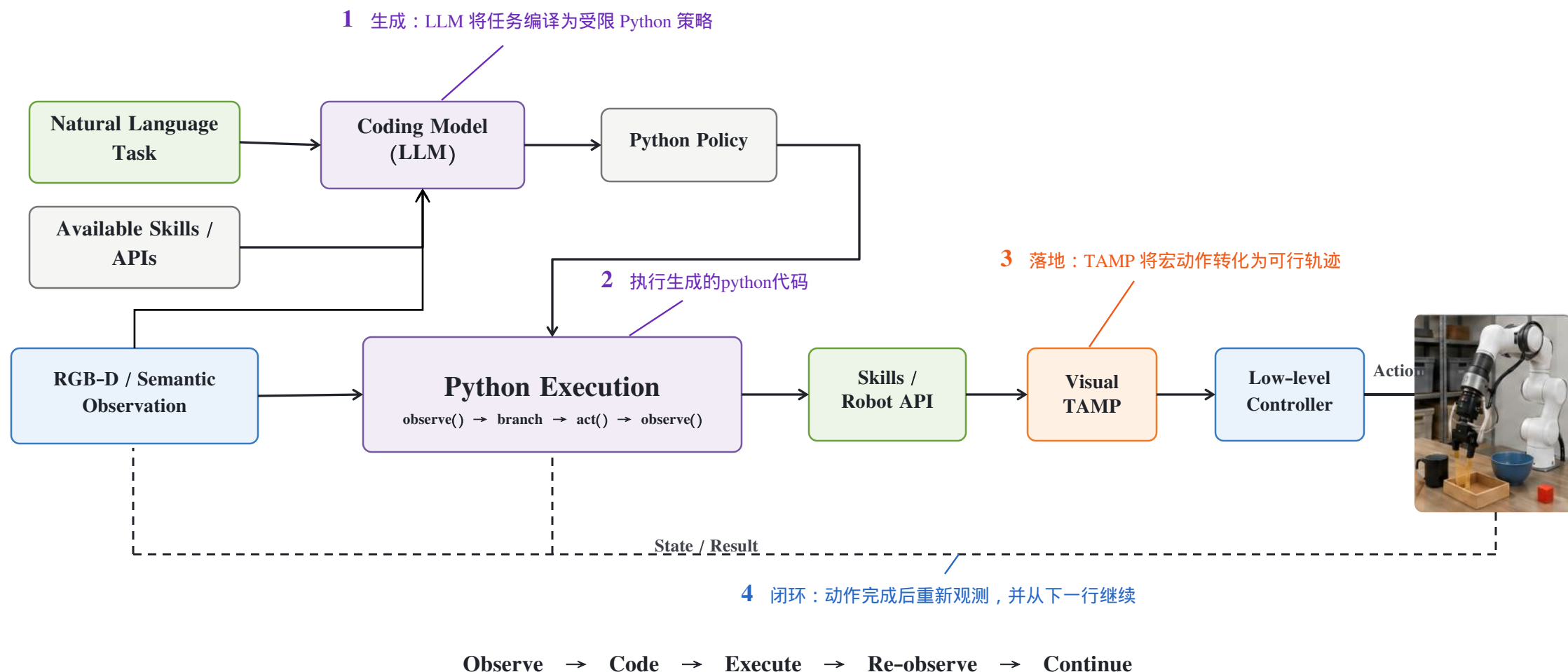
Observe → Plan → Execute → Verify

Traditional Feedback Control



共同结构：目标 → 决策 → 行动 → 环境 → 反馈

CaP:Online Code Generation & Execution

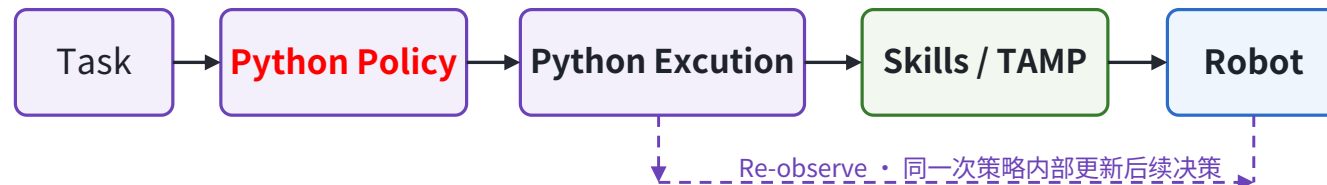


CaP 的核心是将语言任务转化为可执行的代码策略；代码在执行过程中可调用感知与机器人 API，根据当前状态动态决定后续行为。

CaP:Online Code Generation & Execution

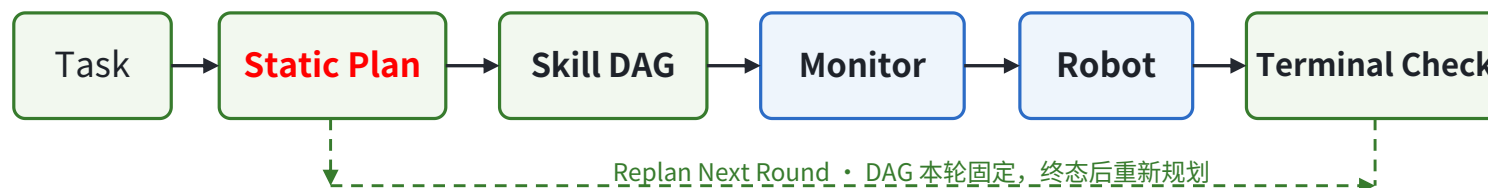
CaP

在线代码执行



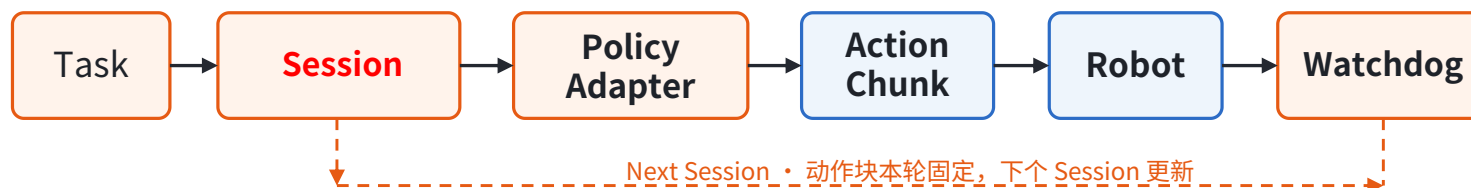
HoloAgent-0

静态计划 + 重规划



PhyAgentOS

动作块 + Session



CaP 通过LLM生成**可执行代码**作为核心策略，在执行过程中结合新观测持续更新后续决策

HoloAgent-0通过 LLM 生成**结构化 Skill DAG**，本轮按固定计划执行，并在终态检查后重新规划

PhyAgentOS通过 Policy Adapter 生成**Action Chunk**，以 **Session** 为单位执行，并在下一 Session 更新动作

Comparative Experiments on Dynamic Tasks

MyCaP

1. Original CaP

Franka / Robosuite / LIBERO
General Python policy
Original perception / execution stack

2. MyCaP Adaptation

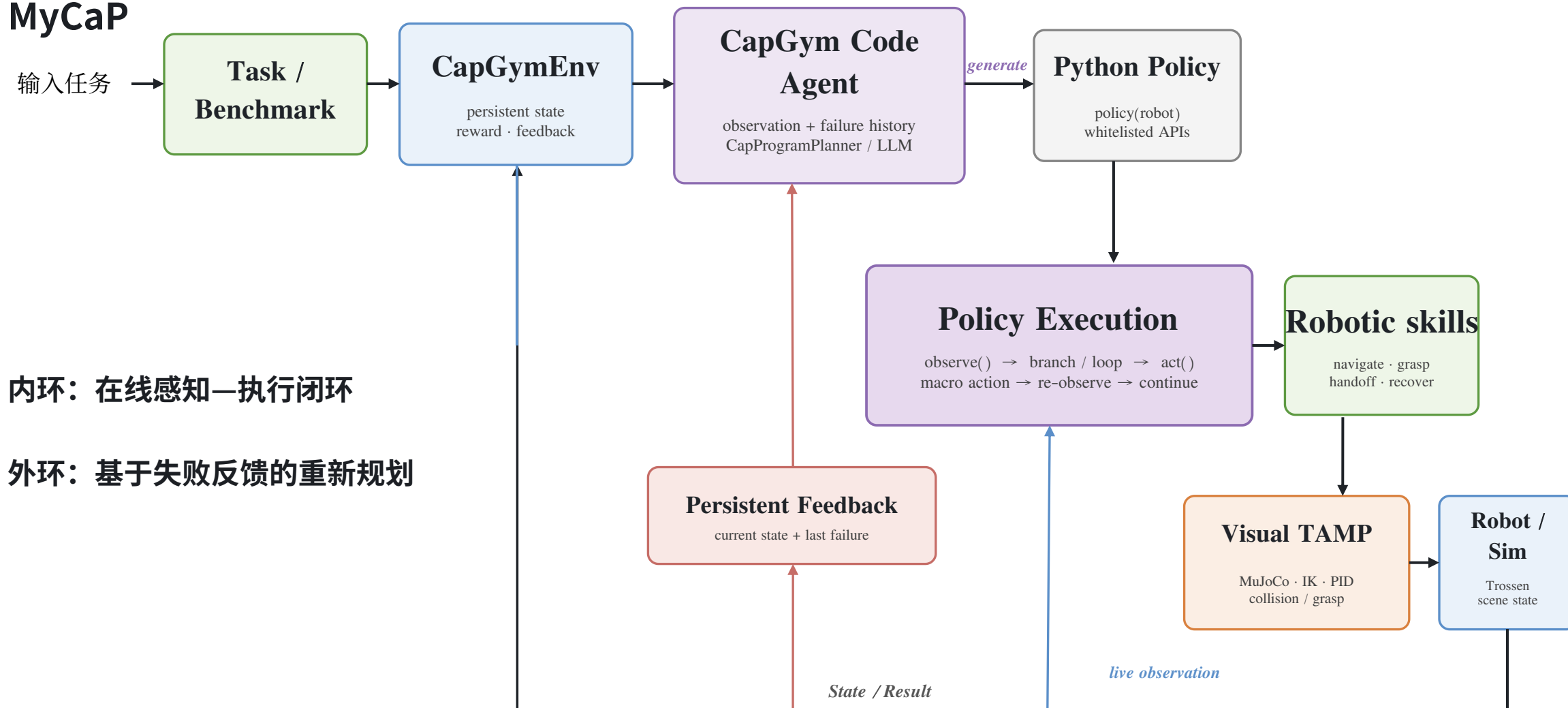
Trossen + MuJoCo	Whitelisted APIs
policy(robot)	Local Skills / TAMP

3. Not Reproduced

VLM feedback translation
Agentic RL

Comparative Experiments on Dynamic Tasks

MyCaP

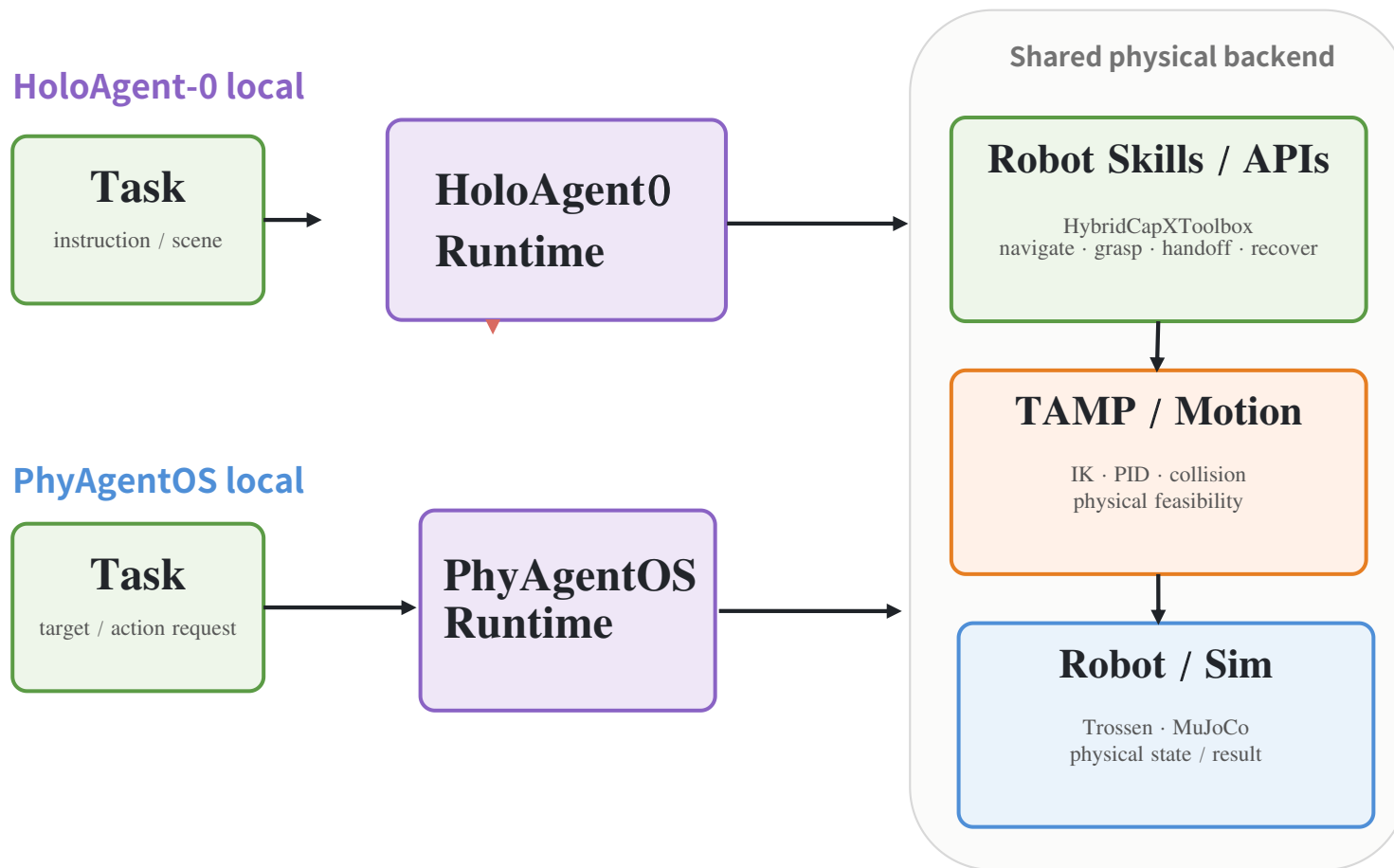


每轮生成策略代码，执行过程中基于实时观测、执行结果和持久状态持续调整后续行为

Comparative Experiments on Dynamic Tasks

HoloAgent-0与PhyAgentOS本地化部署适配

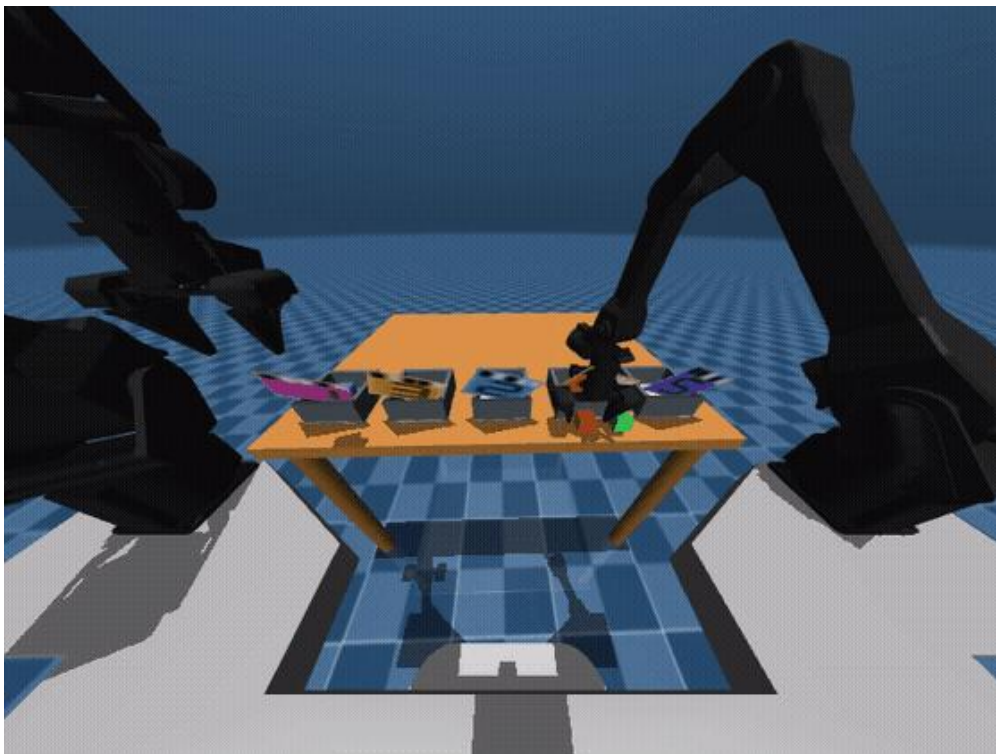
原因：需要相同的操作平台进行测试，需要对env, skill, TAMP, sensor, API修改



共性：HoloAgent-0和PhyAgentOS 都未实现 CaP 式的 in-policy 在线闭环更新

Comparative Experiments on Dynamic Tasks

实验：动态视觉分拣



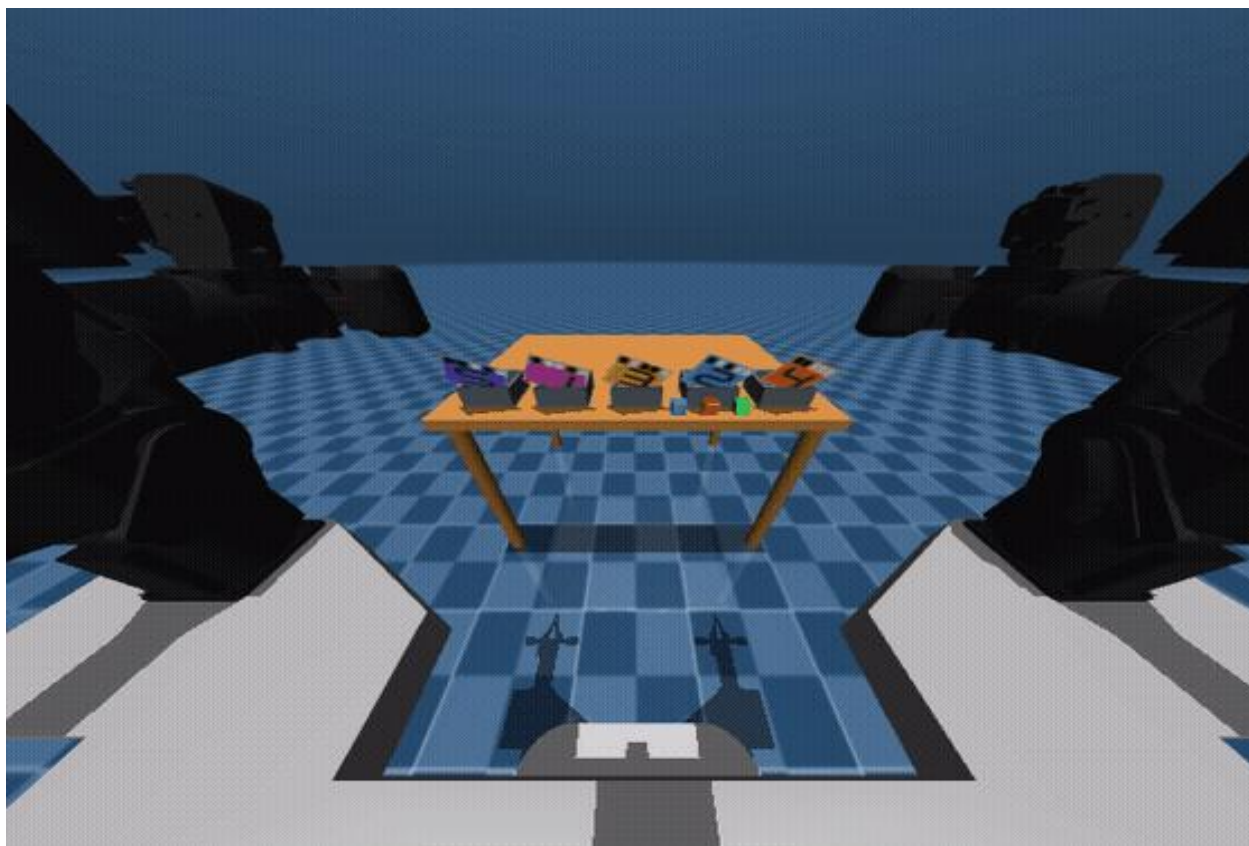
任务：将蓝色方块放入5号盒子，红色方块放入4号盒子，绿色方块放入3号盒子

但是每次执行一次之后，标签会发生变化

进行对照实验，分别用CAP,HOLO,PHY进行多轮测试

Comparative Experiments on Dynamic Tasks

CaP



首策略成功100%

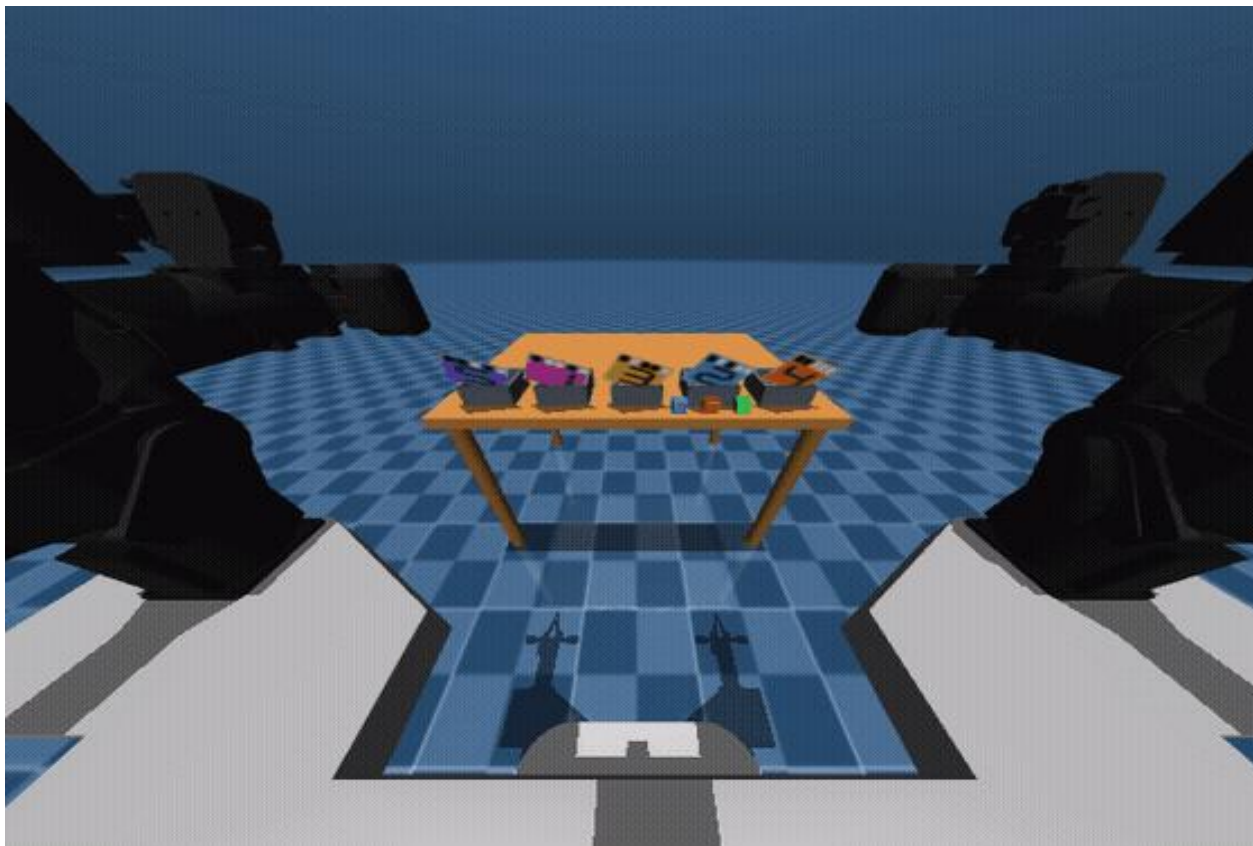
规划 1 次

平均 34.33 个基础动作

程序在每次抓取前重新读取桶标签，因此后续目标不会过期。

Comparative Experiments on Dynamic Tasks

HoloAgent-0



首策略成功 0%

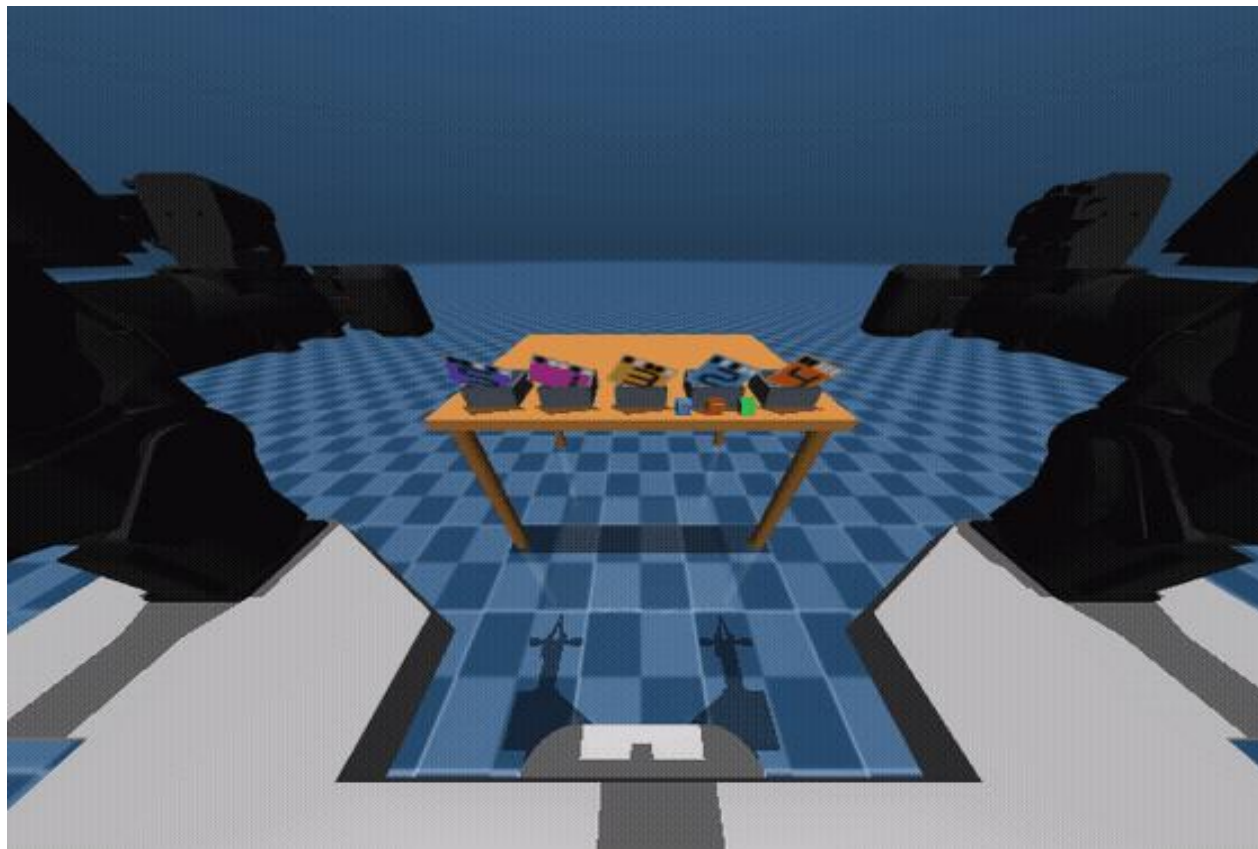
规划 3 次

平均 68.67 个基础动作

workflow DAG 能监控并恢复任务，但
本轮静态动作块不会因语义变化而中途
改写

Comparative Experiments on Dynamic Tasks

PhyAgentOS



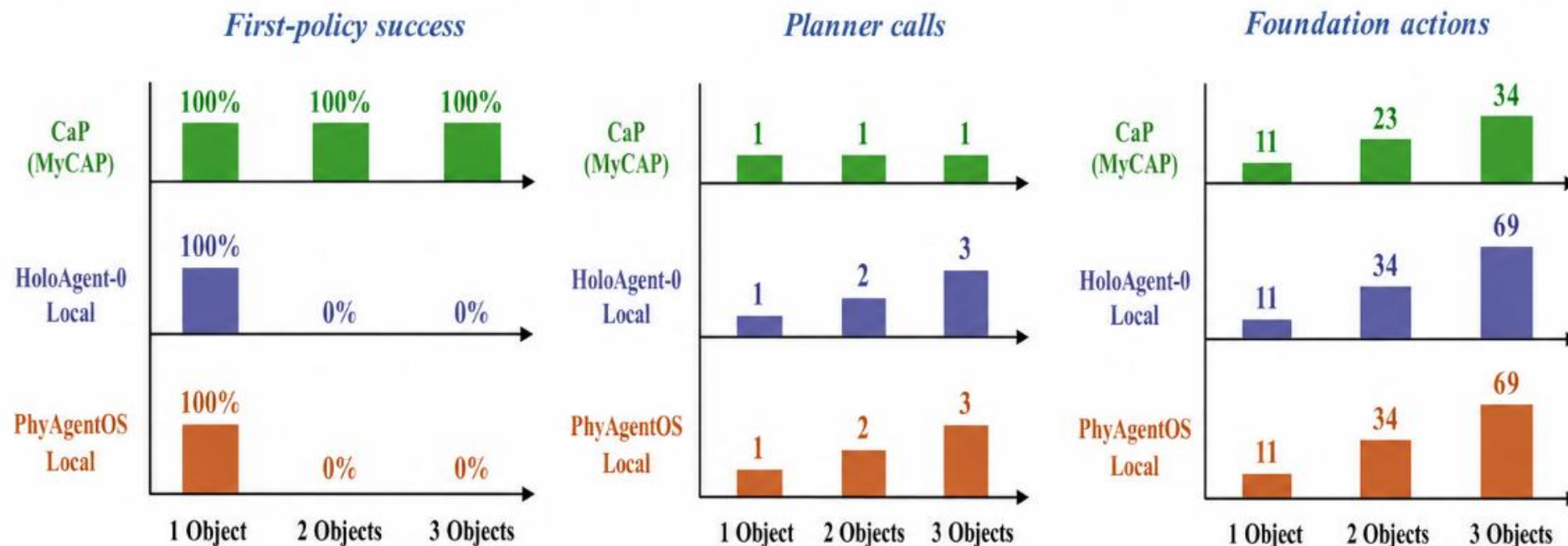
首策略成功 0%

规划 3 次

平均 68.67 个基础动作

adapter、bridge 和 watchdog 保证
执行边界，但仍需下一轮观察来更新动作块。

Comparative Experiments on Dynamic Tasks



最终成功率

三者 100%

三决策首策略

CaP 100% 其余 0%

三决策规划次数

1 / 3 / 3

三决策动作数

34.3 / 68.7 / 68.7

Experimental Results, Contributions & Limitations

100% Same Final Success

→ but

Planner Calls

1 vs 3 / 3

Foundation Actions

34.3 vs 68.7 / 68.7

结论：在本组动态任务中，MyCaP 以相同最终成功率，用更少的高层规划调用和基础动作完成任务。

结果表明，在需要执行过程中重新观测并更新后续决策的任务中，CaP 式在线代码策略表现出更强的在线适应能力。

Experimental Results, Contributions & Limitations



- 1** 在线代码执行 生成 `policy(robot)` , 执行过程中通过 `observe()` 获取新状态 , 并继续进行 `branch / loop` $\rightarrow$ `act()`
- 2** 动态视觉任务 构造动作后标签变化的实验 , 直接检验在线绑定能力。
- 3** HoloAgent-0 /
PhyAgentOS
本地化适配 CaP、HoloAgent-0 Local、PhyAgentOS Local 共享环境、技能与成功判据。
- 4** 对比实验结果 比较首策略成功率、规划次数和基础动作数

Experimental Results, Contributions & Limitations

Current Limitation

Small Benchmark

Structured Feedback Only/no VLM

No Agentic RL

Restricted Python Policy

Local Adaptation

Next Step

More Tasks / Random Seeds

VLM Semantic Feedback

Learning from Success / Failure

More Skills / VLA Policy

Broader Evaluation。